Full Stack Development with MERN

AgriTech

Project Documentation

Empowering farmers through digital technology

Project status: Functional prototype with React frontend and Express REST API

Documentation date: 27 October 2026

1. Introduction

Project title: AgriTech - Smart Farm Management Workspace

Team members: Project team (roles: product analysis, UI/UX, frontend, backend, database, and testing). The repository does not contain individual member names, so names should be added by the submitting team.

AgriTech is a responsive web application that connects farm management, crop monitoring, weather information, soil readings, agricultural resources, community discussions, and administration in one workspace.

2. Project Overview

Purpose. The product helps farmers organize farm data and make more informed day-to-day decisions. It also provides an administrative workflow for products, bookings, users, and farm monitoring data.

Implemented features.

- Authenticated farmer and administrator workspaces with role-based navigation.

- Overview dashboard with farm area, crop count, weather panel, activity, and recent orders.

- Farm creation and farm listing; crop records and crop health progress view.

- Soil monitoring for moisture, pH, nitrogen, phosphorus, and potassium plus farm alerts.

- Weather workspace with city search and optional live OpenWeatherMap integration.

- Resource catalogue for seeds, fertilizer, and pest-control products, with booking actions.

- Community discussions persisted through the API and an expert-consultation entry point.

- Admin centre for user listing, overview counts, booking listing, and booking status updates.

- Responsive layout for mobile, tablet, and desktop screens.

3. Architecture

Frontend. React 18 with Vite and a single application entry point in `src/main.jsx`. The UI is organized into dashboard, farms, crops, soil, weather, resources, community, and admin views. Shared API calls are centralized in `src/api.js`; visual styling is in `src/styles.css`. Lucide React provides icons.

Backend. Node.js and Express use an MVC-style structure: routes map HTTP requests, controllers contain business logic, middleware handles authentication/errors, and model definitions abstract persistence. Helmet, CORS allowlisting, JSON parsing, and rate limiting are registered in `server/src/app.js`.

Database. Mongoose schemas are provided for User, Farm, Crop, Product, Booking, SoilReading, Alert, and Discussion. The application can use MongoDB through `MONGODB_URI`; when MongoDB is not configured it uses an in-memory store and seeds demo users/data.

Main relationships: a user owns farms and bookings; crops reference a farm; bookings reference products; soil readings and alerts reference farms; discussions store the author identity.

4. Setup Instructions

Prerequisites: Node.js (LTS recommended), npm, and optionally MongoDB. A modern browser is required. An OpenWeatherMap API key is optional for live weather.

Installation:

- From the project root, run: npm install

- Enter the backend folder and run: cd server, then npm install

- Create server/.env from server/.env.example. Configure MONGODB_URI for persistence, JWT_SECRET for production, CORS_ORIGIN for the frontend origin, and PORT when a custom port is needed.

- Optionally create a root .env file containing VITE_API_URL and VITE_OPENWEATHER_API_KEY.

The default frontend API target is http://localhost:5000/api. The backend falls back to an in-memory store when MongoDB is unavailable, which is suitable for demonstrations but not for durable production data.

5. Folder Structure

Client / frontend:

src/main.jsx - React application shell and page components.
src/api.js - authenticated API client and weather integration.
src/styles.css - responsive application styling.
index.html - Vite HTML entry point.

Server / backend:

server/src/app.js - Express application, middleware, route mounting, and startup.
server/src/config/database.js - MongoDB connection setup.
server/src/config/store.js - in-memory data store and demo seed data.
server/src/models/schemas.js - Mongoose schemas and model adapters.
server/src/controllers/ - authentication, dashboard, resources, bookings, monitoring, community, and admin logic.
server/src/routes/ - modular route definitions.
server/src/middleware/auth.js - JWT authentication and role authorization.
server/src/middleware/errors.js - not-found and error handling.
server/test/api.test.js - Node test runner API regression tests.

6. Running the Application

Frontend terminal:

```
npm run dev
```

Open the Vite URL shown in the terminal, normally http://localhost:5173.

Backend terminal:

```
cd server
npm start
```

The API listens on http://localhost:5000 by default. Health check: GET /api/health.

Validation:

```
cd server
npm test
```

The backend test script starts the API on port 5101 and runs the API regression suite.

7. API Documentation

Base URL: http://localhost:5000/api. Protected requests use the header `Authorization: Bearer <JWT>`. JSON bodies are used for POST and PUT requests.

Method	Endpoint	Auth / role	Purpose
GET	/health	Public	Service and database status.
POST	/auth/register	Public	Register a farmer account.
POST	/auth/login	Public	Validate credentials and return JWT plus user.
GET	/farms	JWT	List farms visible to the authenticated user.
POST	/farms	JWT	Create a farm.
GET	/farms/:id	JWT	Get one farm with ownership checks.
PUT	/farms/:id	JWT	Update an owned farm.
DELETE	/farms/:id	JWT/admin	Delete a farm.
GET	/crops	JWT	List crops visible to the user.

Method	Endpoint	Auth / role	Purpose
POST	/crops	JWT	Create a crop.
GET/PUT/DELETE	/crops/:id	JWT	Read, update, or remove a crop.
GET	/products	JWT	List resource catalogue products.
POST/PUT	/products[/:id]	Admin	Create or update a product.
DELETE	/products/:id	Admin	Remove a product.
GET	/bookings	JWT	List the user's bookings.
POST	/bookings	JWT	Create a product booking.
PUT	/bookings/:id	Admin	Update booking status.

Method	Endpoint	Auth / role	Purpose
GET	/dashboard	JWT	Return dashboard summary.
GET/POST	/monitoring/soil	JWT	List or submit soil readings.
GET	/monitoring/alerts	JWT	List farm alerts.
PUT	/monitoring/alerts/:id/resolve	JWT	Resolve an alert.
GET/POST	/community	JWT	List or create a discussion.
GET	/admin/overview	Admin	Return administrative counts.
GET	/admin/users	Admin	List users.
PUT	/admin/users/:id	Admin	Update a user.
GET	/admin/bookings	Admin	List all bookings.
PUT	/admin/bookings/:id	Admin	Update an administrative booking.

Example login response:

```
{"token":"<JWT>","user":{"id":"...","name":"Ravi
Kumar","email":"farmer@agritech.local","role":"farmer"}}
```

Example soil request:

```
{"farmId":"farm-1","moisture":62,"ph":6.5,"nitrogen":42,"phosphorus":28,"potassium":35,"note
s":"Weekly reading"}
```

8. Authentication

The API uses stateless JSON Web Tokens. A user registers or logs in through `/api/auth`; the server verifies the password with bcrypt and signs a token with the configured JWT secret. The frontend stores the token in localStorage under `agritech_token` and sends it as a Bearer token on protected API requests.

The authentication middleware verifies the token and attaches the user identity to the request. The authorization middleware restricts administrative actions to users with role `admin`. Resource controllers apply ownership checks for farmer-owned farms and crops. In production, use a strong secret, HTTPS, a restricted CORS origin, and an external persistent database.

Demo in-memory accounts: farmer@agritech.local / password123 and admin@agritech.local / password123. These credentials are for local demonstration only.

9. User Interface

The UI is a responsive dashboard with a left workspace navigation, top account bar, page-intro sections, cards, tables, modal forms, and toast notifications. The role determines whether the Admin centre is visible.

- Overview: KPI cards, weather summary, crop health, activity, and recent orders.

- My farms: farm records and add-farm modal.

- Crop health: crop stage, progress, and attention indicators.

- Soil & alerts: readings submission, readings list, and alert resolution.

- Weather: city search, current conditions, metrics, and forecast; demo conditions remain available without an API key.

- Resources: product catalogue and booking action.

- Community: discussion list and create-discussion form.

- Admin centre: overview counts, users, and order status management.

10. Testing

The backend uses Node.js built-in test runner with an API-level regression suite. The suite starts the application, logs in seeded farmer and admin accounts, and verifies representative end-to-end behavior.

- A farmer can create a crop and receives HTTP 201.

- Out-of-range soil moisture is rejected with HTTP 400.

- A booking above available product stock is rejected with HTTP 400.

- An admin can list bookings and update a booking status to confirmed.

Run the suite with `cd server; npm test`. The frontend currently has no dedicated automated component test suite; manual responsive and API integration checks are recommended for UI changes.

11. Screenshots or Demo

The running product itself is the demo surface. Start both services, sign in with one of the local demo accounts, and open the Vite URL. The following views should be captured for a submission screenshot set:

- Farmer Overview showing KPI cards and current farm activity.

- Crop health and Soil & alerts showing monitoring workflows.

- Weather and Resources showing external-data fallback and booking flow.

- Community showing persisted discussion data.

- Admin centre showing role-based access and booking administration.

Live weather requires VITE_OPENWEATHER_API_KEY. Without that setting, the application deliberately displays demo conditions and explains how to enable live data.

12. Known Issues

- The fallback in-memory store resets data whenever the backend restarts; MongoDB is required for persistence.

- The frontend has a small amount of demo presentation data (for example default dashboard labels) while API data loads.

- The current weather client calls the OpenWeatherMap current-weather endpoint; a full multi-day forecast requires additional provider integration.

- The product brief mentions multilingual support, but the current implementation is English-only.

- No image/GIF capture is stored in this repository; screenshots for final presentation must be captured from the running UI.

- The default demo passwords must be replaced and JWT_SECRET must be configured before production deployment.

13. Future Enhancements

- Add multilingual UI and localized units/date formats for regional farmer communities.

- Add a real notification channel for pest, disease, weather, and irrigation alerts.

- Add richer crop analytics, historical charts, yield prediction, and exportable reports.

- Complete multi-day weather forecasts and geolocation-based farm weather.

- Add image upload for crop disease evidence and expert consultation messaging.

- Add pagination, search, audit logs, password reset, refresh tokens, and stronger production observability.

- Add frontend unit/component tests and browser end-to-end tests.

- Add database indexes, deployment configuration, and automated CI checks.

Conclusion. AgriTech currently delivers a coherent full-stack farm-management prototype: a responsive React workspace, a modular Express API, JWT-based access control, MongoDB-compatible models, an in-memory demo mode, and regression coverage for core API workflows.